

```

"use client";

import React, {
  useEffect,
  useRef,
  createContext,
  useContext,
  forwardRef,
  useMemo,
} from "react";
import { Check, Minus } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🏠 LOCAL TYPE ISOLATION GATEWAY
export interface CheckboxProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange" | "value"
> {
  label?: string;
  error?: string;
  isIndeterminate?: boolean;
  value?: string;
  checked?: boolean;
  onChange?: (checked: boolean | "indeterminate", value?: string) => void;
}

export interface CheckboxGroupProps {
  value: string[];
  onChange: (value: string[]) => void;
  maxChoice?: number;
  disabled?: boolean;
  error?: string;
  children: React.ReactNode;
  className?: string;
}

interface CheckboxGroupContextValue {
  value: string[];
  onChange: (value: string[]) => void;
  maxChoice?: number;
  disabled?: boolean;
  error?: string;
}

```

```
const CheckboxGroupContext = createContext<CheckboxGroupContextValue | null>(
  null,
);
```

```
const Checkbox = forwardRef<HTMLInputElement, CheckboxProps>(
  (
    {
      label,
      error,
      isIndeterminate = false,
      value,
      checked,
      onChange,
      className,
      disabled,
      ...props
    },
    ref,
  ) => {
    const inputRef = useRef<HTMLInputElement>(null);
    const groupContext = useContext(CheckboxGroupContext);
    const isInGroup = !!groupContext;
    const isGroupDisabled = groupContext?.disabled ?? false;
    const isGroupError = groupContext?.error;
    const groupValue = groupContext?.value ?? [];
    const groupMaxChoice = groupContext?.maxChoice;
    const groupOnChange = groupContext?.onChange;

    const isActuallyDisabled = disabled || isGroupDisabled;
    const isChecked = isInGroup
      ? groupValue.includes(value ?? "")
      : (checked ?? false);
    const hasError = error || isGroupError;
    const isAtMaxChoice =
      groupMaxChoice !== undefined &&
      groupValue.length >= groupMaxChoice &&
      !isChecked;

    useEffect(() => {
      if (inputRef.current) {
        inputRef.current.indeterminate = isIndeterminate;
      }
    }, [isIndeterminate]);

    const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
      if (isActuallyDisabled || isAtMaxChoice) return;

      const newChecked = e.target.checked;
      const newValue = value ?? "";
```

```

    if (isInGroup && groupOnChange) {
      const newGroupValue = newChecked
        ? [...groupValue, newValue]
        : groupValue.filter((v) => v !== newValue);
      groupOnChange(newGroupValue);
    }

    onChange?.(newChecked ? true : false, newValue);
  };

  const checkboxClasses = cn(
    "relative inline-flex items-center justify-center",
    "w-5 h-5 shrink-0",
    "rounded-md border-2",
    "border-border bg-background",
    "transition-all duration-200 ease-out",
    "active:scale-95",
    "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring",
    "focus-visible:ring-offset-2 focus-visible:ring-offset-background",
    "disabled:pointer-events-none disabled:opacity-50",
    "disabled:cursor-not-allowed",
    "aria-invalid:border-destructive aria-invalid:ring-3",
    "aria-invalid:ring-destructive/20",
    isChecked && !isIndeterminate
      ? "bg-brand-primary border-brand-primary text-white shadow-sm",
    "hover:shadow-md",
    isIndeterminate
      ? "bg-brand-primary border-brand-primary text-white shadow-sm",
    "hover:shadow-md",
    : "hover:border-brand-primary/50 hover:bg-brand-primary/50",
    isAtMaxChoice && "opacity-50 cursor-not-allowed",
    hasError && "border-destructive focus-visible:ring-destructive",
    className,
  );

  const iconClasses = cn(
    "transition-all duration-200 ease-out",
    isChecked && !isIndeterminate
      ? "scale-100 opacity-100"
      : "scale-0 opacity-0",
    isIndeterminate ? "scale-100 opacity-100" : "scale-0 opacity-0",
  );

  const indeterminateIconClasses = cn(
    "transition-all duration-200 ease-out",
    isIndeterminate ? "scale-100 opacity-100" : "scale-0 opacity-0",
  );

  return (
    <label

```

```

className={cn(
  "inline-flex items-center gap-2 cursor-pointer select-none",
  isActuallyDisabled &&
    "opacity-50 pointer-events-none cursor-not-allowed",
)}
)}
>
<input
  ref={el => {
    inputRef.current = el;
    if (ref) {
      if (typeof ref === "function") {
        ref(el);
      } else {
        ref.current = el;
      }
    }
  }}
  type="checkbox"
  checked={isChecked}
  disabled={isActuallyDisabled || isAtMaxChoice}
  onChange={handleChange}
  aria-checked={isIndeterminate ? "mixed" : isChecked}
  aria-invalid={hasError ? "true" : "false"}
  aria-disabled={isActuallyDisabled}
  aria-required={props.required}
  className="sr-only peer"
  {...props}
/>
<div className={checkboxClasses} aria-hidden="true">
  {isIndeterminate ? (
    <Minus
      className={cn("w-3 h-3", indeterminateIconClasses)}
      aria-hidden="true"
    />
  ) : (
    <Check
      className={cn("w-3.5 h-3.5", iconClasses)}
      aria-hidden="true"
    />
  )}
</div>
{label && (
  <span
    className={cn(
      "text-sm font-medium leading-none",
      "transition-colors duration-200 ease-out",
      "text-text-main",
      hasError && "text-destructive/90",
      isActuallyDisabled && "opacity-50",
    )}
  )}

```

```

        >
        {label}
      </span>
    )}
  </label>
);
},
);
Checkbox.displayName = "Checkbox";

export const CheckboxGroup = ({
  value,
  onChange,
  maxChoice,
  disabled,
  error,
  children,
  className,
}: CheckboxGroupProps) => {
  const contextValue = useMemo(
    () => ({
      value,
      onChange,
      maxChoice,
      disabled,
      error,
    }),
    [value, onChange, maxChoice, disabled, error],
  );

  return (
    <CheckboxGroupContext.Provider value={contextValue}>
      <div
        className={cn("inline-flex flex-col gap-2", className)}
        role="group"
        aria-invalid={!error}
        aria-disabled={disabled}
      >
        {children}
        {error && (
          <p
            className={cn(
              "text-sm",
              "text-destructive/90",
              "transition-colors duration-200 ease-out",
              "mt-1",
            )}
            role="alert"
            aria-live="polite"
          >

```

```
        {error}
      </p>
    )}
  </div>
</CheckboxGroupContext.Provider>
);
};
CheckboxGroup.displayName = "CheckboxGroup";
```